

Musterlösung zur Musterklausur Programmierung 2

Daniel Appenmaier, Juni 2026

Aufgabe 1

Klasse *Terminator*

```
@Data // 1
@EqualsAndHashCode(callSuper = true)
@ToString(callSuper = true)
public class Terminator extends Roboter implements Comparable<Terminator> { // 1,5

    private final Series series; // 0,5

    public Terminator(String serialNumber, LocalDate fabricationDate, Series series) { // 0,5
        super(serialNumber, fabricationDate); // 1
        this.series = series; // 0,5
    } // 2

    @Override
    public int compareTo(Terminator other) { // 0,5
        return Double.compare(other.series.getWeightInKg(), series.getWeightInKg()); // 1,5
    } // 2

} // 7
```

Klasse *RoboterFactory*

```
public record RoboterFactory<T extends Roboter>(Map<String, T> robots) { // 1,5

    public Optional<T> getRoboterBySerialNumber(String serialNumber) { // 0,5
        for (Entry<String, T> e : robots.entrySet()) { // 1
            if (e.getKey().equals(serialNumber)) { // 1
                return Optional.of(e.getValue()); // 1
            }
        }
        return Optional.empty(); // 0,5
    } // 4

    public List<T> getRobotersByFabricationYear(int fabricationYear) { // 0,5
        List<T> robotsByFabricationYear = new ArrayList<>(); // 0,5
        for (T t : robots.values()) { // 1
            if (t.getFabricationDate().getYear() == fabricationYear) { // 1,5
                robotsByFabricationYear.add(t); // 0,5
            }
        }
        return robotsByFabricationYear; // 0,5
    } // 4,5

} // 10
```

Aufgabe 2

Klasse *RoboterFactoryTest*

```
public class RoboterFactoryTest { // 0,5

    @Mock // 0,5
    Terminator t800; // 0,5
    @Mock // 0,5
    Terminator t1000; // 0,5
    RoboterFactory<Terminator> factory; // 0,5

    @BeforeEach // 0,5
    void setup() { // 0,5
        MockitoAnnotations.openMocks(this); // 1

        factory = new RoboterFactory<>(new HashMap<>()); // 0,5
        factory.robots().put("800-1", t800); // 1
        factory.robots().put("1000-1", t1000); // 1
    } // 4,5

    @Test // 0,5
    void testGetRoboterBySerialNumber() { // 0,5
        assertEquals(Optional.empty(), factory.getRoboterBySerialNumber("1000-0")); // 1,5
        assertEquals(Optional.of(t1000), factory.getRoboterBySerialNumber("1000-1")); // 1,5
    } // 4

    @Test // 0,5
    void testGetRobotersByFabricationYear() { // 0,5
        when(t800.getFabricationDate()).thenReturn(LocalDate.parse("2029-03-24")); // 1
        when(t1000.getFabricationDate()).thenReturn(LocalDate.parse("2032-07-12")); // 1
        assertEquals(t800, factory.getRobotersByFabricationYear(2029)
            .getFirst()); // 1,5
    } // 4,5
} // 16
```

Aufgabe 3

Klasse *TerminatorQueries*

```
public record TerminatorQueries(List<Terminator> terminators) { // 1

    public List<String> getAllUniqueSeriesDescriptions() { // 0,5
        List<String> series; // 0,5
        series = terminators.stream() // 0,5
            .map(t -> t.getSeries().getDescription()) // 0,5
            .distinct() // 0,5
            .toList(); // 0,5
        return series; // 0,5
    } // 3,5

    public List<Terminator> getAllHumanoidTerminatorsSortedByFabricationDate() { // 0,5
        List<Terminator> sortedT1000Terminators; // 0,5
        sortedT1000Terminators = terminators.stream() // 0,5
            .filter(t -> t.getSeries().isHumanoid()) // 1
            .sorted((t1, t2) -> t2.getFabricationDate().compareTo(t1.getFabricationDate()))
            // 1,5
            .toList(); // 0,5
        return sortedT1000Terminators; // 0,5
    } // 5

    public double getAverageWeightInKg() throws Exception { // 1
        OptionalDouble averageWeightInKg; // 0,5
        averageWeightInKg = terminators.stream() // 0,5
            .mapToDouble(t -> t.getSeries().getWeightInKg()) // 1
            .average(); // 0,5
        if (averageWeightInKg.isEmpty()) { // 1
            throw new Exception(); // 1
        }
        return averageWeightInKg.getAsDouble(); // 0,5
    } // 6

    public Map<Double, List<Terminator>> getTerminatorsByWeightInKg() { // 0,5
        Map<Double, List<Terminator>> terminatorsByWeightInKg; // 0,5
        terminatorsByWeightInKg = terminators.stream() // 0,5
            .collect(Collectors.groupingBy(t -> t.getSeries().getWeightInKg())); // 1,5
        return terminatorsByWeightInKg; // 0,5
    } // 3,5
} // 19
```